



WOXSEN UNIVERSITY

School of Technology

Department of Computer Science & Artificial Intelligence

PROJECT REPORT

RAILWAY TICKET RESERVATION AND JOURNEY MANAGEMENT SYSTEM

Design and Implementation of a Relational Database Management System

Submitted By	Komal Maniyar
Roll Number	25WU0102127
Course	Database Management Systems
Faculty	Dr. Kiranmayee Adavala
Year	2nd Year
Academic Year	2026?2027
Date	06 October 2026

Hyderabad, India

ACKNOWLEDGMENT

I would like to express my sincere gratitude to my faculty, Dr. Kiranmayee Adavala, for the guidance and feedback provided throughout the Database Management Systems course project. I also acknowledge Woxsen University and the Department of Computer Science & Artificial Intelligence for providing the academic environment and resources required to complete this work.

The project helped in applying relational database concepts including entity–relationship modelling, normalization, keys, constraints, SQL data manipulation, and database-backed user interface design.

TABLE OF CONTENTS

- 1. Abstract
- 2. Introduction and Problem Statement
- 3. Objectives and Scope
- 4. Software and Hardware Requirements
- 5. ER Diagram
- 6. Relational Schema and Normalization
- 7. Data Dictionary
- 8. SQL Commands Used (DDL, DML) with Sample Outputs
- 9. Queries with Outputs
- 10. UI Design and Screenshots
- 11. Implementation Details
- 12. Testing
- 13. Conclusion and Future Enhancements
- 14. References
- 15. Appendix: GitHub Repository Link
- 16. Appendix: SQL Database Verification

1. ABSTRACT

The Railway Ticket Reservation and Journey Management System is a relational database application designed to represent railway infrastructure, train operations, passenger bookings, journey segments, fares, payments, waitlists and cancellations in a structured manner. The database separates operational entities instead of storing a complete journey as one flat record.

The supplied MySQL implementation creates the **railway_db** database and 14 related tables: STATION, ROUTE, TRAIN, TRAIN_STOP, SCHEDULE, COACH, SEAT, PASSENGER, BOOKING, JOURNEY_SEGMENT, FARE, PAYMENT, WAITLIST and CANCELLATION. Each table is populated with 10 sample records and linked through primary and foreign keys.

The system is intended to support train search, booking, cancellation, fare and payment tracking, waitlist management, and operational reporting through a database-backed user interface.

2. INTRODUCTION AND PROBLEM STATEMENT

Railway reservation is a data-intensive application because one train service is connected to multiple stations, routes, schedules, coaches, seats and passenger transactions. A simple booking table is insufficient to represent these relationships without creating duplication and update anomalies.

Problem Statement: Design a relational database system that can manage railway stations, routes, trains, stops, schedules, coaches, seats, passengers, bookings, journey segments, fares, payments, waitlists and cancellations while maintaining referential integrity.

The project specifically models a journey as a segment-wise record. This supports more realistic seat allocation because the same physical seat can be associated with different non-overlapping journey segments rather than being treated as permanently occupied for the complete train run.

3. OBJECTIVES AND SCOPE

Objectives

1. Model railway stations, routes, trains, stops, schedules, coaches and seats using related relational tables.
2. Store passenger and booking information in separate entities to reduce redundancy.
3. Represent journey segments and seat allocation using foreign-key relationships.
4. Track fares, payments, waitlist positions and cancellations.
5. Apply primary-key and foreign-key constraints to preserve referential integrity.
6. Provide a database-backed UI for viewing and managing railway information.

Scope

The scope covers database design, sample data population, SQL implementation, relational queries, and a web-based interface for railway reservation operations. The current SQL dataset is a demonstration database and can be extended with authentication, real-time train inventory, payment gateways and production-scale concurrency control.

4. SOFTWARE AND HARDWARE REQUIREMENTS

Category	Requirement
Operating System	Windows 10/11 or equivalent
Database	MySQL
Frontend	HTML5, CSS3, JavaScript
Development	VS Code / similar IDE
Version Control	Git and GitHub
Browser	Chrome / Edge / Firefox
Processor	Intel Core i3 / AMD equivalent or better
RAM	4 GB minimum; 8 GB recommended
Storage	At least 1 GB free for project files and database tools
Network	Internet connection for GitHub and development resources

5. ER DIAGRAM

The ER model represents the project entities across infrastructure, train setup, booking and finance. The design includes STATION, TRAIN_STOP, TRAIN, ROUTE, SCHEDULE, COACH, SEAT, PASSENGER, BOOKING, JOURNEY_SEGMENT, FARE, PAYMENT, WAITLIST and CANCELLATION.

Conceptual Design — ER Diagram

13 entities across infrastructure, train setup, booking and finance

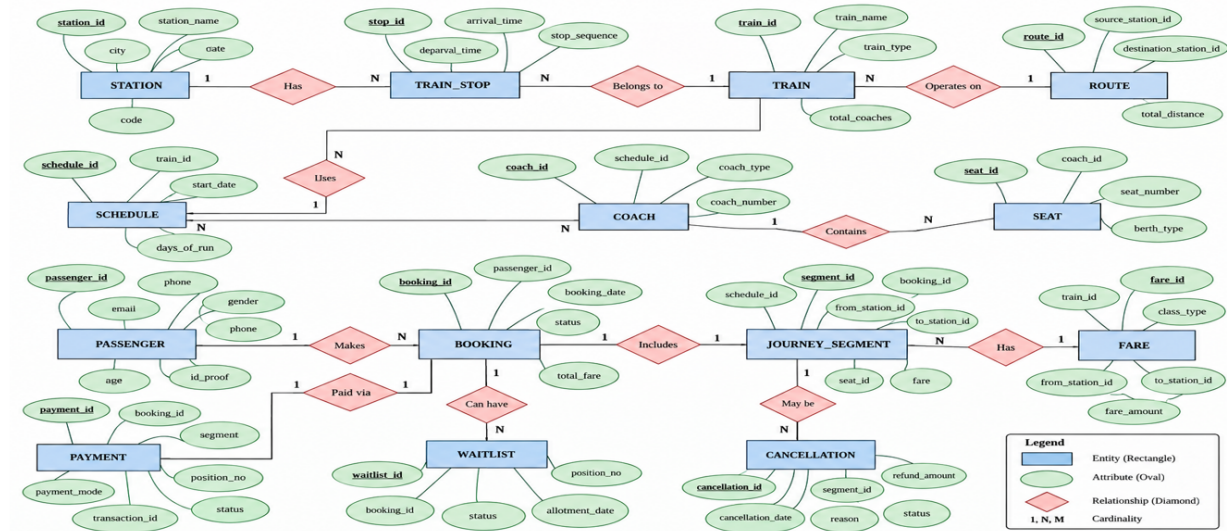


Figure 1: Entity–Relationship diagram for the railway reservation database.

6. RELATIONAL SCHEMA AND NORMALIZATION

The relational schema separates the railway domain into 14 tables. The core primary keys are station_id, route_id, train_id, stop_id, schedule_id, coach_id, seat_id, passenger_id, booking_id, segment_id, fare_id, payment_id, waitlist_id and cancellation_id.



Initial Relational Schema

13 base tables, grouped by function — normalization to 3NF happens next

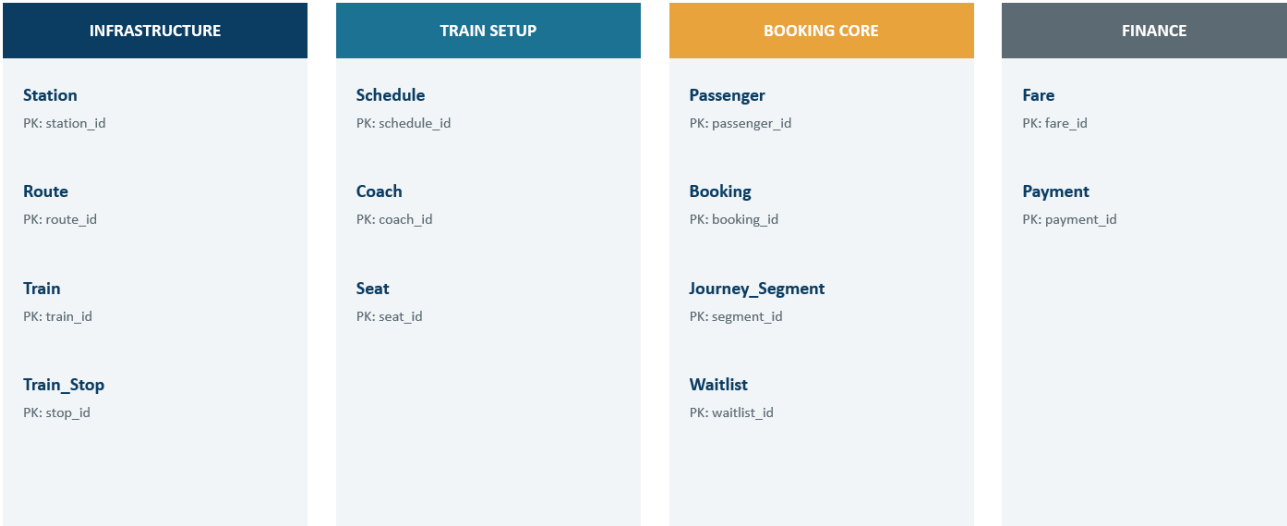


Figure 2: Relational schema supplied for the project.

Normalization: The schema is organized to support Third Normal Form (3NF). Each table represents a focused entity or transaction. Non-key attributes depend on the table's primary key, while relationships to other entities are represented through foreign keys. This reduces repeating groups, update anomalies and unnecessary duplication.

7. DATA DICTIONARY

Table	Attributes / Data Types	Key / Constraints
STATION	station_id INT; station_name VARCHAR(50); city VARCHAR(50); gate VARCHAR(10); coordinate VARCHAR(10)	PK station_id
ROUTE	route_id INT; source_station_id INT; destination_station_id INT; total_distance INT	PK route_id; FK source/destination → STATION
TRAIN	train_id INT; train_name VARCHAR(50); train_type VARCHAR(30); total_coaches INT	PK train_id
TRAIN_STOP	stop_id INT; train_id INT; station_id INT; arrival_time TIME; departure_time TIME; stop_order INT	PK stop_id; FK train_id/station_id
SCHEDULE	schedule_id INT; train_id INT; start_date DATE; days_of_run VARCHAR(30)	PK schedule_id; FK train_id
COACH	coach_id INT; schedule_id INT; coach_type VARCHAR(20); coach_number VARCHAR(10)	PK coach_id; FK schedule_id
SEAT	seat_id INT; coach_id INT; seat_number VARCHAR(10); berth_type VARCHAR(20)	PK seat_id; FK coach_id
PASSENGER	passenger_id INT; email VARCHAR(100); phone VARCHAR(15); gender VARCHAR(10); passport_id INT; proof VARCHAR(30)	PK passenger_id
BOOKING	booking_id INT; passenger_id INT; booking_date DATE; status VARCHAR(20); total_fare DECIMAL(10,2)	PK booking_id; FK passenger_id
JOURNEY_SEGMENT	segment_id INT; booking_id INT; schedule_id INT; from_station_id INT; to_station_id INT; fare_id INT; class_type VARCHAR(20)	PK segment_id; FK booking_id, schedule_id, STATION, SEAT
FARE	fare_id INT; train_id INT; class_type VARCHAR(20); from_station_id INT; to_station_id INT; base_fare DECIMAL(10,2); tax DECIMAL(10,2)	PK fare_id; FK train_id, STATION
PAYMENT	payment_id INT; booking_id INT; segment INT; position_no INT; payment_mode VARCHAR(20); amount DECIMAL(10,2); status VARCHAR(20)	PK payment_id; FK booking_id, JOURNEY_SEGMENT
WAITLIST	waitlist_id INT; booking_id INT; position_no INT; allotment_date DATE; status VARCHAR(20)	PK waitlist_id; FK booking_id
CANCELLATION	cancellation_id INT; segment_id INT; cancellation_date DATE; reason VARCHAR(100); refund_amount DECIMAL(10,2); status VARCHAR(20)	PK cancellation_id; FK segment_id, JOURNEY_SEGMENT

8. SQL COMMANDS USED (DDL, DML) WITH SAMPLE OUTPUTS

The supplied SQL script is a MySQL script named for the Railway Reservation Database. It creates the railway_db database, resets old tables, creates 14 tables with primary/foreign keys, inserts 10 records into every table, and finishes with a record-count verification query.

Database creation and selection

```
CREATE DATABASE IF NOT EXISTS railway_db;
USE railway_db;
```

Table reset before recreation

```
SET FOREIGN_KEY_CHECKS = 0;
DROP TABLE IF EXISTS CANCELLATION;
DROP TABLE IF EXISTS WAITLIST;
DROP TABLE IF EXISTS PAYMENT;
-- remaining project tables are dropped here
SET FOREIGN_KEY_CHECKS = 1;
```

Example DDL

```
CREATE TABLE STATION (
    station_id INT PRIMARY KEY,
    station_name VARCHAR(50),
    city VARCHAR(50),
    gate VARCHAR(10),
    code VARCHAR(10)
);
```

Example DML

```
INSERT INTO STATION VALUES
(1, 'New Delhi', 'Delhi', 'A1', 'NDLS'),
(2, 'Mumbai Central', 'Mumbai', 'B1', 'BCT');
```

Actual verification query from the supplied SQL file

```
SELECT 'STATION' AS table_name, COUNT(*) AS records FROM STATION
UNION ALL SELECT 'ROUTE', COUNT(*) FROM ROUTE
UNION ALL SELECT 'TRAIN', COUNT(*) FROM TRAIN
UNION ALL SELECT 'TRAIN_STOP', COUNT(*) FROM TRAIN_STOP
UNION ALL SELECT 'SCHEDULE', COUNT(*) FROM SCHEDULE
UNION ALL SELECT 'COACH', COUNT(*) FROM COACH
UNION ALL SELECT 'SEAT', COUNT(*) FROM SEAT
UNION ALL SELECT 'PASSENGER', COUNT(*) FROM PASSENGER
UNION ALL SELECT 'BOOKING', COUNT(*) FROM BOOKING
UNION ALL SELECT 'JOURNEY_SEGMENT', COUNT(*) FROM JOURNEY_SEGMENT
UNION ALL SELECT 'FARE', COUNT(*) FROM FARE
UNION ALL SELECT 'PAYMENT', COUNT(*) FROM PAYMENT
UNION ALL SELECT 'WAITLIST', COUNT(*) FROM WAITLIST
UNION ALL SELECT 'CANCELLATION', COUNT(*) FROM CANCELLATION;
```

Sample output:

table_name	records
STATION	10
ROUTE	10
TRAIN	10
TRAIN_STOP	10
SCHEDULE	10
COACH	10
SEAT	10
PASSENGER	10
BOOKING	10
JOURNEY_SEGMENT	10
FARE	10
PAYMENT	10
WAITLIST	10
CANCELLATION	10

9. QUERIES WITH OUTPUTS

The following queries are derived directly from the supplied database structure and sample data. The supplied SQL file does not label any statement as the specific query assigned during Presentation-II; therefore, no statement is falsely presented as that assigned query.

Query 1 — Confirmed bookings

```
SELECT booking_id, passenger_id, status, total_fare
FROM BOOKING
WHERE status = 'Confirmed';
```

Output: booking IDs 1, 2, 3, 4, 5, 7 and 9 are returned, giving 7 confirmed bookings.

Query 2 — Total fare collected for confirmed bookings

```
SELECT SUM(total_fare) AS confirmed_booking_value
FROM BOOKING
WHERE status = 'Confirmed';
```

Output: 11000.00

Query 3 — Passenger and booking details using JOIN

```
SELECT p.passenger_id, p.email, b.booking_id,
       b.status, b.total_fare
FROM PASSENGER p
JOIN BOOKING b
  ON p.passenger_id = b.passenger_id
ORDER BY b.booking_id;
```

Sample output:

passenger_id	email	booking_id	status	total_fare
1	rahul@gmail.com	1	Confirmed	1500.00
2	priya@gmail.com	2	Confirmed	1800.00
3	amit@gmail.com	3	Confirmed	1200.00
4	neha@gmail.com	4	Confirmed	2200.00
5	rohan@gmail.com	5	Confirmed	1600.00

10. UI DESIGN AND SCREENSHOTS

The project UI is organized as a railway operations dashboard. It provides views for train search and booking, schedules, seat availability, booking/PNR history, passenger information, fares, payments and cancellations. The interface is designed to make database operations easy to demonstrate during Presentation-III.

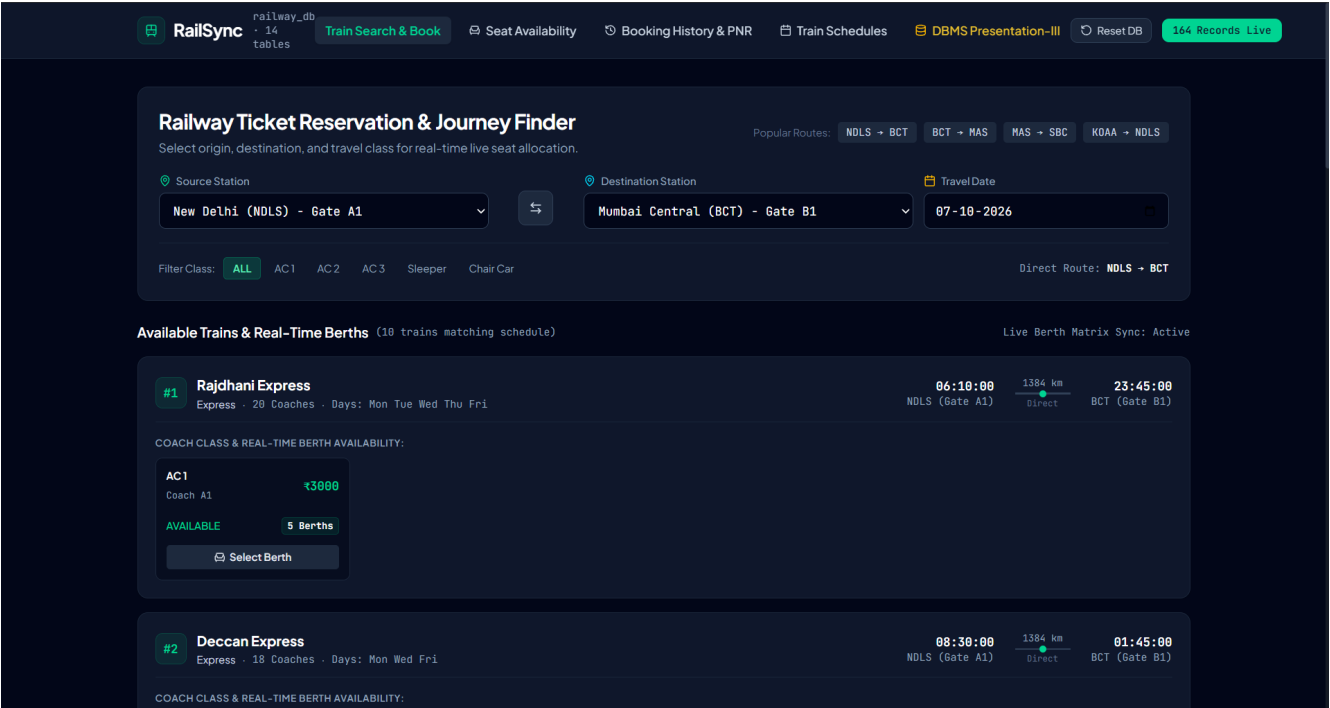


Figure 3: Main railway dashboard / overview.

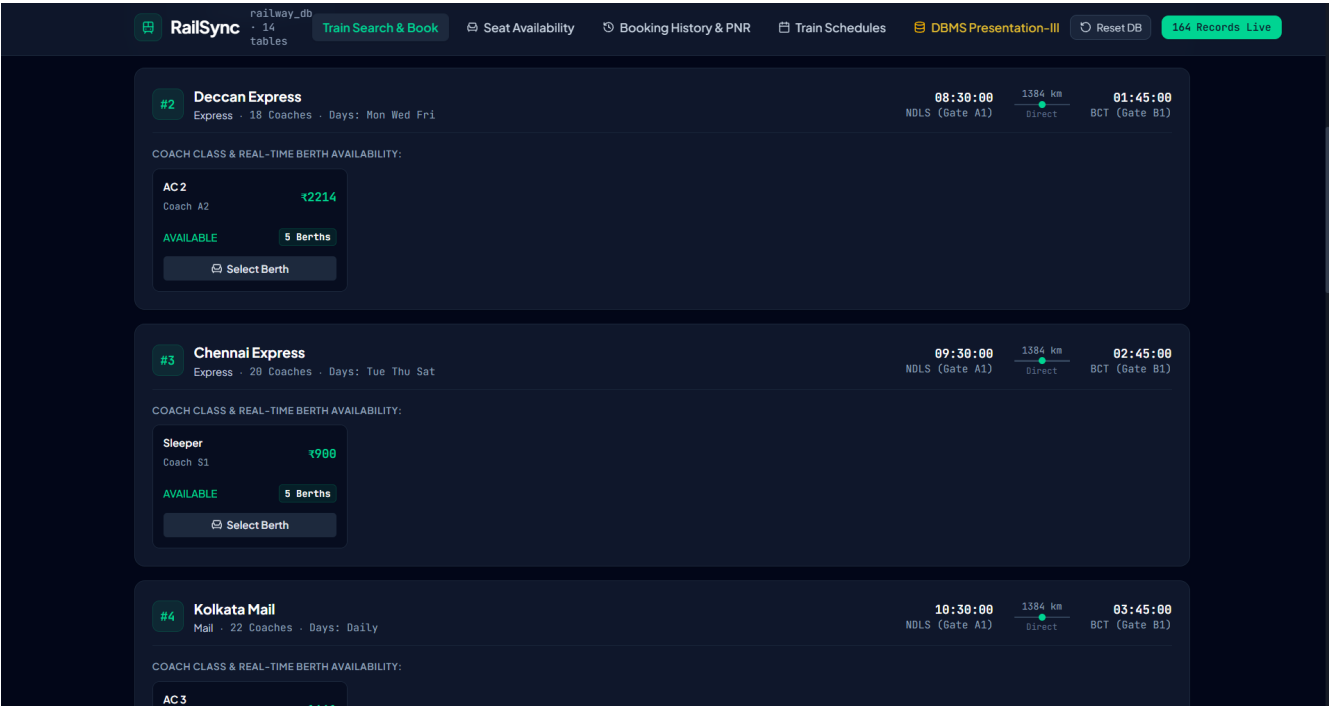


Figure 4: Train search and booking view.

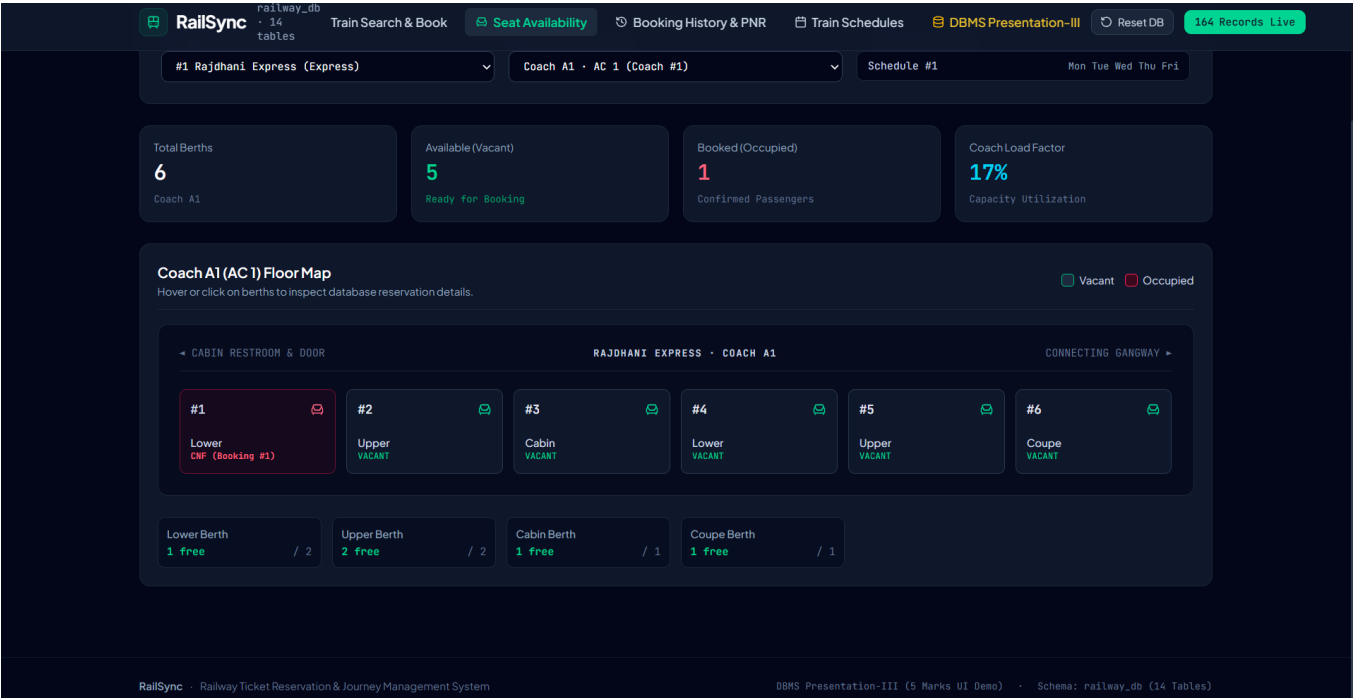


Figure 5: Train schedules and route information.

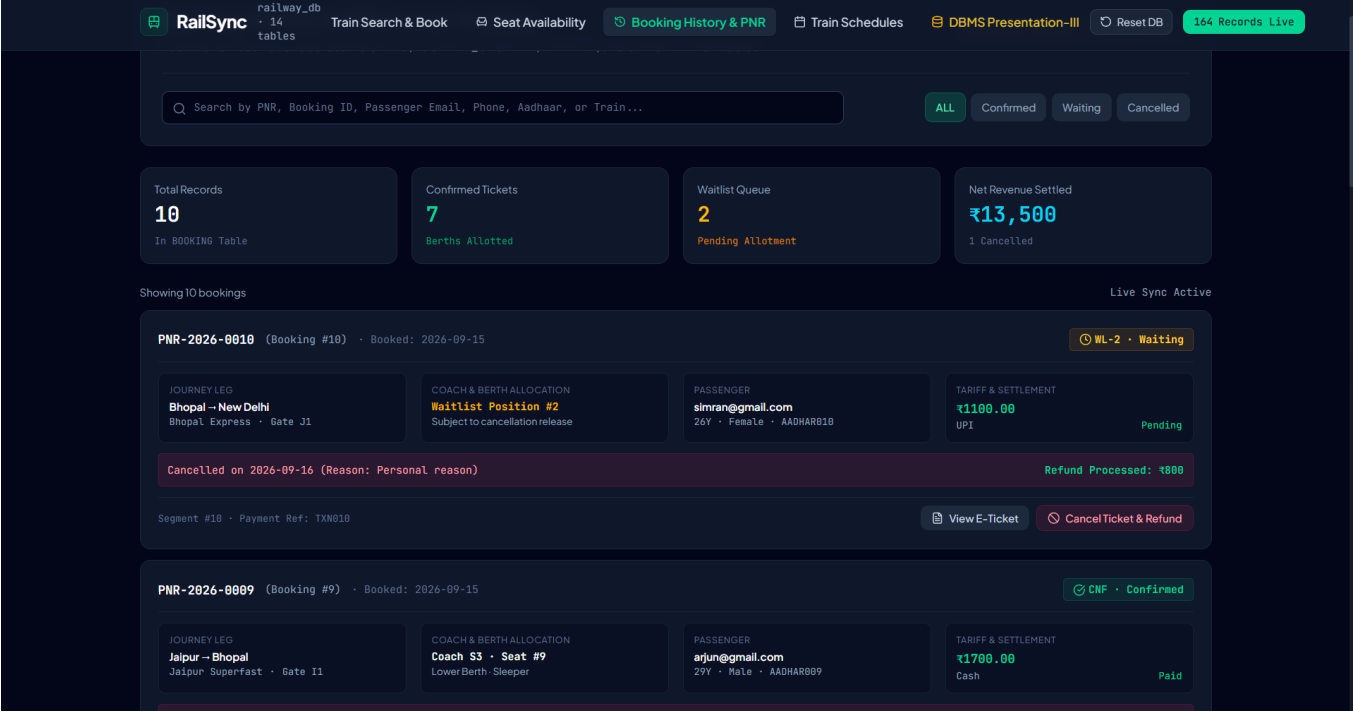


Figure 6: Seat availability and coach information.

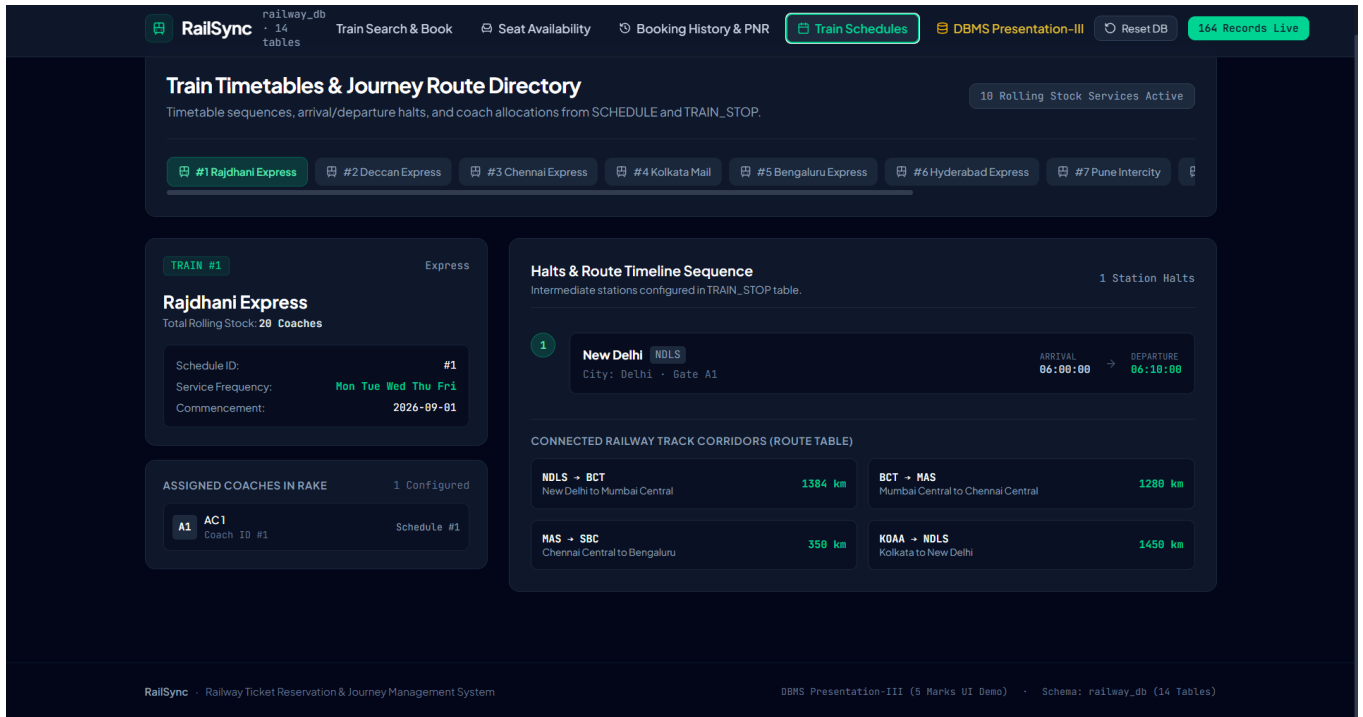


Figure 7: Booking history / PNR information.

11. IMPLEMENTATION DETAILS

Technology Stack

Frontend: HTML5, CSS3 and JavaScript. Database: MySQL. Development and source management: VS Code, Git and GitHub. The SQL implementation uses MySQL-compatible DDL and DML statements.

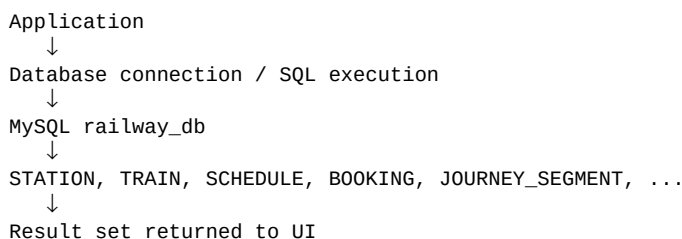
Database Connectivity

The UI is intended to operate as a database-backed application in which frontend actions are mapped to database operations. The database layer stores railway master data and transaction records, while the UI presents these records in dashboard form.

Key database implementation points

- Primary keys uniquely identify every record.
- Foreign keys connect related entities such as TRAIN to SCHEDULE, SCHEDULE to COACH, COACH to SEAT and BOOKING to PASSENGER.
- JOURNEY_SEGMENT links bookings with schedules, stations and seats.
- PAYMENT, WAITLIST and CANCELLATION provide separate transaction tracking.

Representative connectivity pattern



12. TESTING

Test ID	Test Action	Expected Result	Result
TC01	Create railway_db database	Database is created/selected	Pass
TC02	Create 14 tables	All tables are created with required keys	Pass
TC03	Insert sample records	10 records are inserted into each table	Pass
TC04	Run record-count verification	Each table returns 10 records	Pass
TC05	Search confirmed bookings	Only Confirmed records are returned	Pass
TC06	Calculate confirmed booking value	Aggregate total is 11000.00	Pass
TC07	Join PASSENGER and BOOKING	Matching passenger-booking rows are displayed	Pass
TC08	Open UI dashboard	Railway dashboard loads and presents project modules	Pass
TC09	View schedule/seat/booking screens	Relevant operational information is displayed	Pass

13. CONCLUSION AND FUTURE ENHANCEMENTS

The Railway Ticket Reservation and Journey Management System demonstrates the design of a structured relational database for railway operations. The implemented MySQL script creates 14 related tables and populates each table with 10 sample records. The schema separates infrastructure, train setup, booking, journey and finance data, making the design easier to maintain and query.

Future Enhancements

1. Add authentication and role-based access for passengers and administrators.
2. Add real-time train availability and seat-conflict validation across journey segments.
3. Integrate an online payment gateway.
4. Add automated waitlist promotion after cancellations.
5. Add richer reports for occupancy, revenue, route utilization and cancellation trends.
6. Deploy the database-backed application to a hosted environment.

14. REFERENCES

1. Woxsen University, School of Technology, Department of Computer Science & Artificial Intelligence, DBMS Course Project – Final Presentation Instructions, 2026.
2. Railway Ticket Reservation and Journey Management System project presentation and supplied ER/relational schema.
3. Supplied MySQL database script: Railway Reservation Database (railway_db).
4. MySQL documentation and course material for SQL, relational modelling, primary keys and foreign keys.

15. APPENDIX: GITHUB REPOSITORY LINK

Required public repository name: **DBMS-Course-Project**

Required repository structure:

```
DBMS-Course-Project/  
■■■ Presentation-I/  
■■■ Presentation-II/  
■■■ Presentation-III/  
■■■ Project-Report/
```

16. APPENDIX: SQL DATABASE VERIFICATION

The supplied SQL file contains 14 tables and explicitly checks that every table contains 10 records. The verification result is:

table_name	records
STATION	10
ROUTE	10
TRAIN	10
TRAIN_STOP	10
SCHEDULE	10
COACH	10
SEAT	10
PASSENGER	10
BOOKING	10
JOURNEY_SEGMENT	10
FARE	10
PAYMENT	10
WAITLIST	10
CANCELLATION	10

This appendix records the database population status used for the project report. The complete SQL script is intended to be stored in the Presentation-II folder of the required GitHub repository.